

ITAI 1378

Computer Vision for Artificial Intelligence

Module 04

Neural Networks for Computer Vision

When machines stopped following our recipes and started writing their own.



Author: Patricia McManus

Collaborator: Claude (Anthropic)

AI and Robotics Program | Digital and Information Technology Center of Excellence

Houston City College

Welcome to Module 04

Where We Are

In Module 03 you learned how computer vision was built for forty years. Humans designed the feature extractor (HOG, SIFT, LBP, and friends), and the machine only had to learn the last step: mapping feature vectors to labels using a classifier like SVM or Random Forest. That recipe shipped, made money, and still ships in many industries today.

Module 04 introduces the change that broke that recipe open. In 2012, a neural network called AlexNet won the ImageNet competition by a margin so large that within a year, every serious CV team in the world had switched approaches. The change was not that classifiers got better. The change was that machines started learning the feature extractor too, directly from raw pixels, end to end. This module teaches you what neural networks are, how they learn, and why that single shift redrew the entire field.

What You Will Learn

By the end of this module you will be able to:

- Explain the loose biological analogy between real neurons and the artificial neurons inside a neural network.
- Describe the perceptron (the simplest artificial neuron) and the multi layer perceptron (the foundation of every modern network).
- Walk through the training loop in plain English: forward pass, loss, gradient descent, backpropagation.
- Name the main activation functions (ReLU, Sigmoid, Tanh, Softmax) and pick the right one for a given layer.
- Explain why a basic neural network struggles with images and what convolutional networks (next module) will do about it.
- Name the main frameworks (PyTorch, TensorFlow with Keras, Hugging Face) and the standard model interchange format (ONNX).
- Diagnose the most common training problems (overfitting, underfitting, bad learning rate, vanishing gradients) and apply standard fixes.

Why Should You Care

Three reasons. First, neural networks power almost every CV product you interact with daily. Your phone's face unlock. Your social media's photo tagging. Your camera's portrait mode. Your car's parking sensors. The autopilot demos. They are all neural networks under the hood. Understanding this material is no longer optional for a working CV engineer.

Second, the conceptual leap in this module is the leap. Once you internalize the idea that a learnable function can replace a designed function, you can read a modern paper, evaluate a vendor pitch, or design your own pipeline without losing the plot. Everything after this module (CNNs, transformers, generative models, multimodal systems) is variations on the same core idea: more parameters, smarter architectures, the same training loop.

Third, this is where the math sneaks in. Module 03 had no math; Module 04 still avoids equations in this booklet but you will see notation in any framework documentation. The Math Moment callouts in this booklet are optional, but if you skim them, the framework code stops looking like magic.

Part 01 | From Biology to Math

Neural networks borrow their name and their basic structure from the brain. The borrowing is loose. Real neurons are vastly more complex than artificial ones, and the analogy can be misleading if taken too literally. But the inspiration is useful, and the vocabulary is shared, so we start there.

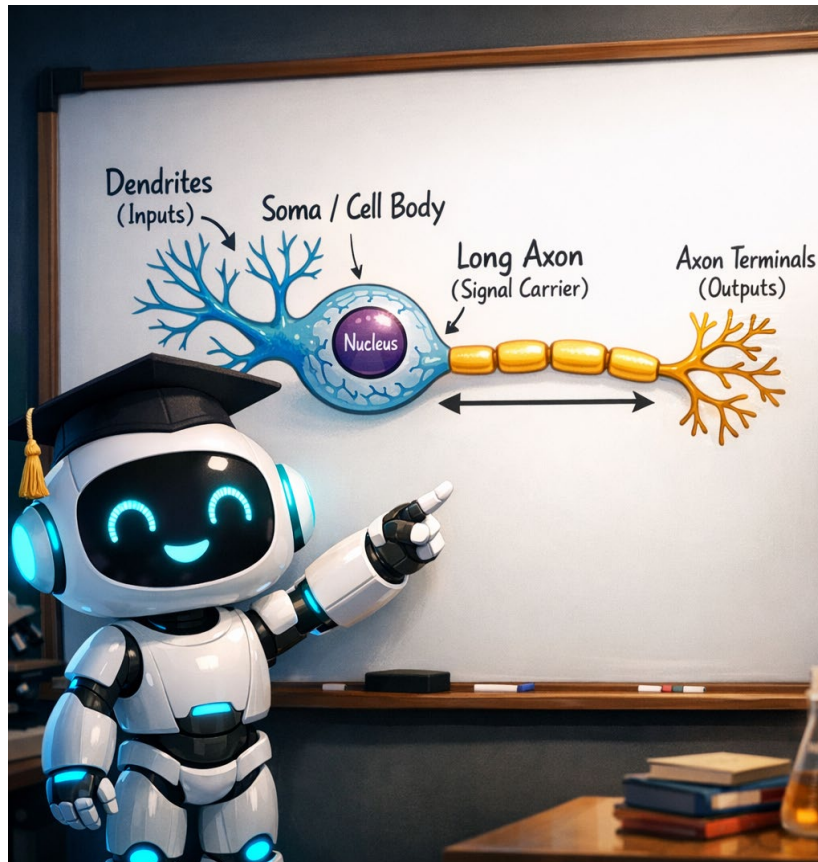
The Biological Neuron

Your brain has roughly eighty six billion neurons. Each one is a cell with a simple job: collect electrical signals from its neighbors, decide whether to fire, and pass a signal forward to other neurons. A single neuron is not intelligent. Intelligence emerges from billions of them working together, in patterns of connections that change over time as you learn.

Three Parts You Need to Know

A biological neuron has three main parts. Dendrites are the branches that receive signals from other neurons. The cell body, including the nucleus, processes those signals and decides whether to fire. The axon is the long thin output cable that carries the signal to the next neuron's dendrites.

Two neurons connect through a synapse, the gap between an axon and a dendrite. When one neuron fires, chemicals cross the synapse and influence the next neuron. Whether the next neuron then fires depends on how many signals it has received and how strong the synaptic connections are. Learning, in a biological sense, happens by changing synaptic strengths. The connections that fire together get stronger. The ones that do not, get weaker.



The Artificial Neuron

An artificial neuron, also called a unit or a node, is a mathematical caricature of a biological one. It has the same logical structure (receive inputs, combine them, produce an output) but the implementation is just arithmetic. The result is something simple enough to run billions of times on a chip and complex enough, in aggregate, to learn anything we throw at it.

What an Artificial Neuron Does

An artificial neuron does three things. First, it collects inputs (numbers coming in from other neurons or from raw data). Second, it weighs and combines them (some inputs matter more than others; each input has a weight, and the neuron adds up all the weighted inputs). Third, it decides whether to fire (if the total is above a threshold, the neuron passes a signal forward; if not, it stays quiet).

That is the entire algorithm. Three steps. Every modern AI model on the planet, no matter how large or impressive, is built out of these three steps applied billions of times. ChatGPT is artificial neurons. AlphaGo is artificial neurons. The neural network that unlocks your phone is artificial neurons. The architecture varies, the scale varies, the data varies. The neuron is the same.

The Loose Biological Analogy

Inputs map to dendrites. Weights map to synaptic strengths. The combine step maps to the cell body. The decide step maps to the firing threshold. The output maps to the axon. Each correspondence is loose; real biology is far more complex. But the structure helps newcomers see what is going on.

MISCONCEPTION ALERT | Brains and networks are not the same

A common myth is that artificial neural networks work the way the human brain does. They do not. Real neurons have timing, spiking, neurotransmitter chemistry, and structural plasticity that artificial neurons lack entirely. The analogy is inspiration, not implementation. Engineers who take the brain analogy too literally make worse design decisions, not better ones.

The Big Shift from Module 03

In Module 03, humans designed the features and the machine learned only the classifier. In Module 04, the machine learns the features too. Here is the structural difference, side by side.

Classic ML (Module 03)

A human engineer chooses the feature extractor (HOG, SIFT, LBP). A human engineer codes how each feature is computed. The features get packed into a vector. Only the classifier (the final mapping from vector to label) is learned from data.

Neural Networks (Module 04 onward)

The network sees raw pixels. The early layers learn what to look for. The middle layers learn how to combine those patterns. The deeper layers learn high level concepts. The final layer makes the prediction. Everything between the input and the output is learned. Human design lives only at the architectural level (how many layers, how many neurons, how they connect), not at the feature level.

This shift is the single largest conceptual change in modern AI. Every advance since 2012, from convolutional networks to transformers to large language models, inherits this idea: learn the whole pipeline end to end. The price was data hunger and compute cost. The prize was an accuracy ceiling that classic ML could not approach.

KEY TAKEAWAY

Neural networks are stacks of simple decision makers (artificial neurons) connected together. The biology is loose inspiration; the math is what matters. The big idea: instead of humans designing features and machines learning classifiers, the network learns both end to end.

THINK ABOUT IT | Where would you draw the line?

In Module 03, the human drew the line between feature design and classifier learning. In Module 04, the machine learns everything from raw pixels to final output. As the boundary shifts further toward the machine, what is left for the engineer to design? Think about this before you read Part 02. You will need an answer when you build your first network.

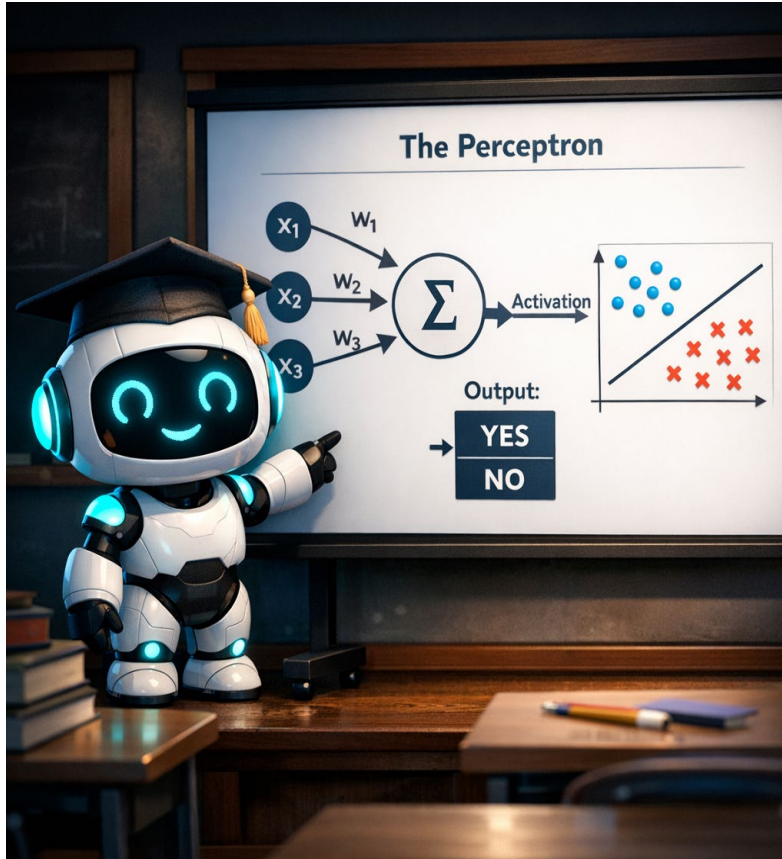
Part 02 | The Perceptron and the Multi-Layer Perceptron

With the biological analogy out of the way, this part builds up the actual structure of a neural network. We start with the smallest unit (the perceptron), see why one is not enough, stack them into layers (the multi layer perceptron), and learn the vocabulary that describes any network you will ever build or read about.

The Perceptron

The perceptron was invented by Frank Rosenblatt at Cornell in 1958. It is the simplest possible artificial neuron and the building block of every modern network. A perceptron takes some numbers in, multiplies each by a weight, adds the results together, adds a bias term, and then applies a simple threshold to decide whether the output should be on or off.

Picture it this way. A perceptron is a tiny voting machine. Each input is a voter. Each weight is how much that voter's opinion counts. The bias is a baseline tilt (some perceptrons are easier to activate than others). The threshold is the rule for when the votes are enough to pass.



Historical Note

The 1958 New York Times reported that the perceptron would soon walk, talk, see, and reproduce itself. That prediction was about sixty years premature, but the underlying idea really is the foundation of everything that followed. Every neuron in every modern model, from your phone's face unlock to ChatGPT, traces back to this 1958 invention.

In 1969, a famous book by Marvin Minsky and Seymour Papert showed that a single perceptron could not solve some very simple problems (the XOR problem is the classic example). That result helped trigger the first AI winter, when funding dried up and the field went quiet. The fix turned out to be obvious in hindsight: stack many perceptrons together. But it took until the 1980s for that idea to mature, and until 2012 for it to fully dominate.

From One Neuron to a Network

A single perceptron can only learn to separate things by a straight line in feature space. Real problems are rarely that simple. Cats and dogs cannot be separated by a straight line in pixel space. Tumors and healthy tissue cannot. Anything interesting requires more expressive power.

The fix is to stack many perceptrons in layers, where the output of one layer feeds into the next. Each layer transforms the data into a more useful representation. With enough layers and enough neurons, a network can represent any pattern. (This is sometimes called the universal approximation property, and it is a real mathematical result, though it does not say how to actually train such a network.)

The Multi-Layer Perceptron

A multi layer perceptron, abbreviated MLP, is a stack of perceptron layers connected in sequence. Every modern network is a variation on this structure. The MLP has three categories of layers: the input layer, one or more hidden layers, and the output layer.

The Input Layer

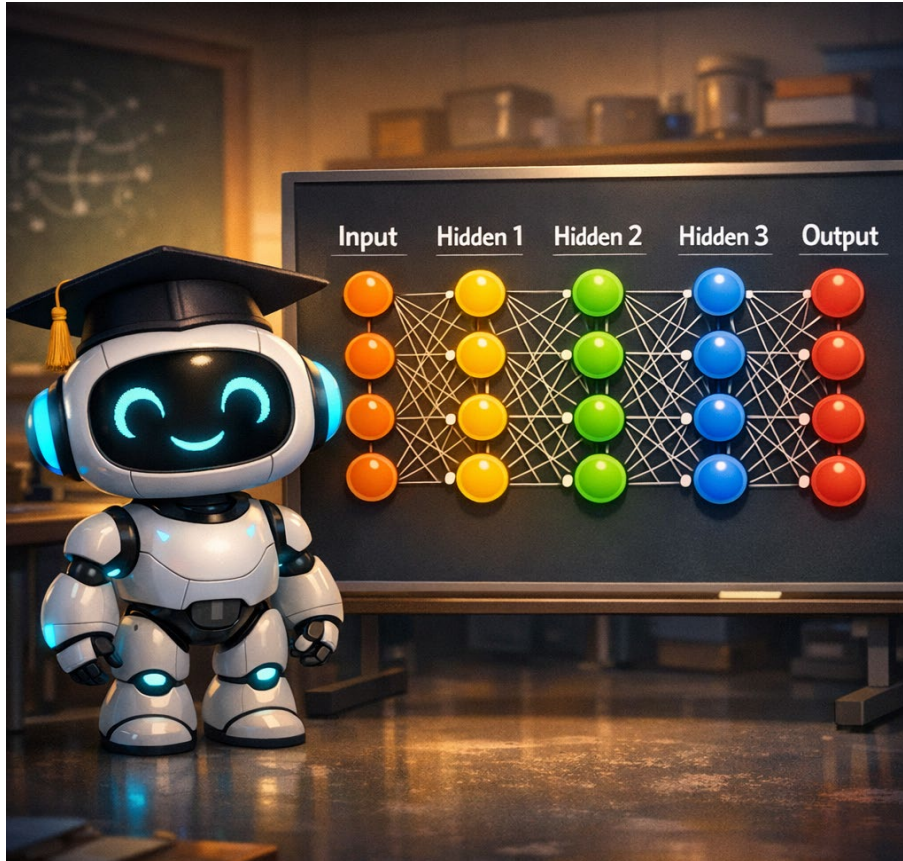
The input layer takes the raw data. For an image, this means the pixel values themselves. The input layer does not transform the data; it just receives it. The size of the input layer is determined by the data. A 28 by 28 grayscale image flattens into 784 numbers, so the input layer has 784 neurons.

Hidden Layers

Hidden layers are where the work happens. Each hidden layer transforms the data into a more useful representation. The early layers detect simple patterns (an edge here, a brightness change there). The middle layers combine those simple patterns into more complex ones (a curve, a corner, a texture). The deepest layers detect the high level concepts (a cat ear, a tumor, a defective solder joint). This hierarchy is not designed; it emerges during training.

The Output Layer

The output layer produces the final answer. For a classifier with ten classes (like the digits 0 through 9), the output layer has ten neurons, each one producing a score for one class. The class with the highest score wins. For a regression problem (predicting a number), the output layer has one neuron producing that number directly.



The Anatomy of a Neural Network

Four pieces of vocabulary describe every neural network. Memorize them. They appear in every framework documentation, every paper, every interview, and every API.

Neurons (Nodes)

The simple decision makers in each layer. Also called units or nodes. The number of neurons in a layer is one of the architectural choices you make when designing a network.

Weights and Biases

The numbers the network learns. Weights set how much each input matters to a given neuron. Biases shift the threshold so the neuron is easier or harder to activate. Weights and biases together are called the network's parameters. A modern model has millions to billions of them.

Activation Functions

The rule each neuron uses to decide what to pass on. Part 04 of this booklet covers them in depth. Common choices are ReLU, Sigmoid, Tanh, and Softmax.

Architecture

How many layers, how many neurons in each, how they connect. The shape of the network. Architecture is designed by humans. Weights and biases are learned from data. That boundary, between what humans choose and what machines learn, is the practical version of the big shift you read about in Part 01.

KNOWLEDGE CHECK | Four words you must know cold

1. Neuron: the basic processing unit. Receives inputs, applies weights and bias, passes the result through an activation function, produces an output.
2. Weight: a learned number that determines how strongly one neuron influences another.
3. Bias: a learned number that shifts the activation threshold of a neuron.
4. Activation function: the rule that turns a neuron's weighted sum into an output. Adds the non linearity that makes the whole network capable of learning complex patterns.

CAREER CONNECTION | What recruiters listen for

When asked 'How does a neural network work?' weak candidates start describing backpropagation. Strong candidates start with 'A neural network is a stack of simple decision makers called neurons, organized into layers. Each neuron has weights and a bias, applies an activation function, and passes its output to the next layer. The network learns by adjusting weights to minimize a loss function.' Then they continue. Vocabulary first, mechanics second.

MATH MOMENT (optional) | The weighted sum in plain English

Inside every neuron, the inputs and weights get combined into a single number called the weighted sum. The math is straightforward. Multiply each input by its weight (a higher weight means that input counts more). Add up all those multiplied values. Add the bias. That is the weighted sum. The activation function then takes this number and decides what the neuron passes forward. You never compute this by hand in practice; PyTorch and TensorFlow do it for you. The point of understanding it is so that 'multiply, add, decide' becomes a phrase you can use confidently when describing networks.

 THINK ABOUT IT | Why does the bias term exist?

Without a bias, every neuron must activate based purely on its weighted sum. That sum is zero when all inputs are zero, which forces the neuron to be off in that case. Adding a bias means the neuron can be tilted on or off regardless of the inputs. Why might that flexibility matter for learning complex patterns? Think about it before reading on; the answer is hidden in plain sight in every network architecture.

 INDUSTRY PRACTICE | Where you will see this vocabulary

When you write PyTorch code, the lines `nn.Linear(784, 128)` and `nn.ReLU()` are the perceptron in disguise. Linear is the weighted sum and bias. ReLU is the activation function. Every neural network library uses these same two operations as the building blocks. Recognize them in unfamiliar code and any model becomes readable.

 KNOWLEDGE CHECK | Quick check on perceptrons and MLPs

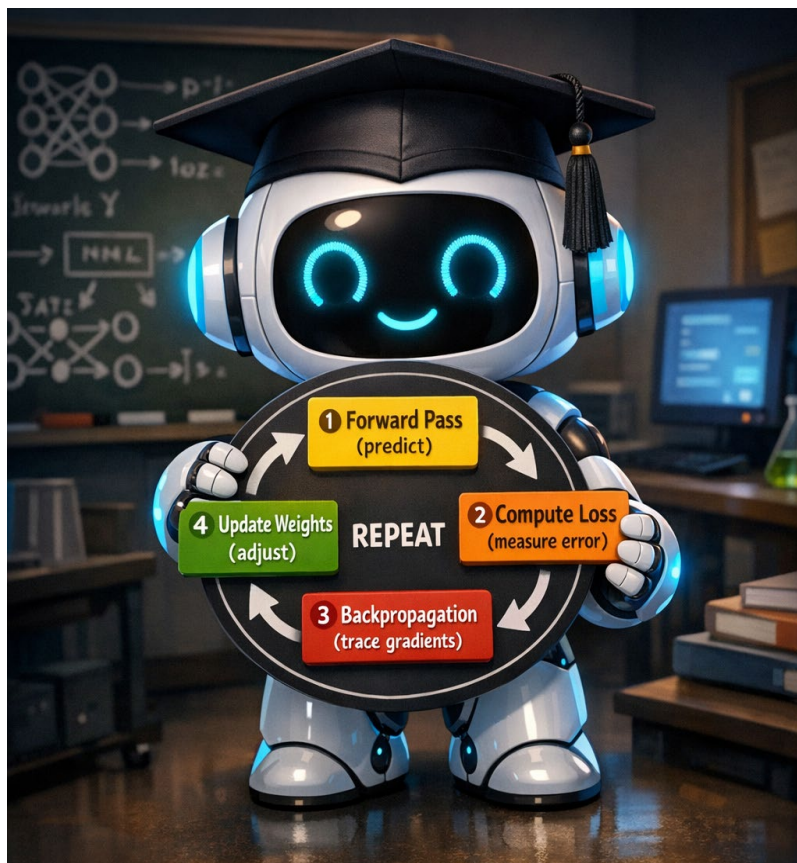
1. Name the three operations a single perceptron performs in order.
2. Why is a single perceptron not enough for problems like recognizing cats versus dogs?
3. What is the difference between architecture (designed by humans) and weights (learned by machines)?

Part 03 | How Networks Learn

A neural network with random weights is useless. It outputs random predictions. The art of deep learning is in the training loop, the process by which the network adjusts its weights so that its predictions become accurate. This part walks you through that loop step by step, in plain English. You will see one technical term per step (forward pass, loss, gradient descent, backpropagation). Each one names a real concept worth understanding.

The Four Step Training Loop

Every neural network trains by repeating four steps over and over with many training examples. The steps are: predict, measure error, trace blame, adjust weights. Repeat millions of times. That is the entire algorithm.



Step 1: The Forward Pass

An image enters the network. Each layer transforms it. The output layer produces a guess. For a cat photo, the output might look like '42 percent cat, 35 percent dog, 23 percent bird'. The network is unsure but has expressed an opinion. The forward pass is just running the network on an input and seeing what it says.

Important detail: the forward pass uses the current weights and biases. If those weights are random (early in training), the forward pass will produce random predictions. If those weights are well trained (after many iterations), the forward pass will produce confident, accurate predictions. The math does not change; only the parameters.

Step 2: The Loss

After the forward pass, you compare the network's guess to the correct answer. The difference between guess and truth is the loss, a single number that measures how wrong the network was. Big loss means very wrong. Small loss means almost right. The whole goal of training is to make the loss as small as possible across all training examples.

Common Loss Functions

Cross entropy loss is the standard for classification. It heavily penalizes confident wrong answers and rewards confident right ones. Mean squared error is the standard for regression (predicting numbers). For a particular task, the choice of loss function matters; pick the wrong one and the network learns the wrong thing.

MATH MOMENT (optional) | Loss in one sentence

Loss is a single number that summarizes how wrong all your predictions are on the training set. The training algorithm tries to minimize this number by changing the network's weights. That is the whole optimization problem.

Step 3: Gradient Descent

To shrink the loss, you need to know which direction to push the weights. That direction is given by the gradient. A gradient, in the loose intuitive sense, is the slope of the loss with respect to each weight. If a weight is contributing to the loss being high, the gradient tells you to reduce that weight. If a weight is helping reduce the loss, the gradient tells you to increase it.

The Blindfolded Hillside

Imagine you are blindfolded on a hillside and want to reach the lowest point in the valley. You feel which way is downhill, take a small step in that direction, feel again, take another step, and keep going until you cannot go any lower. That is gradient descent. The hillside is the loss function. Your position is the network's current weights. Each step changes the weights slightly to reduce the loss.



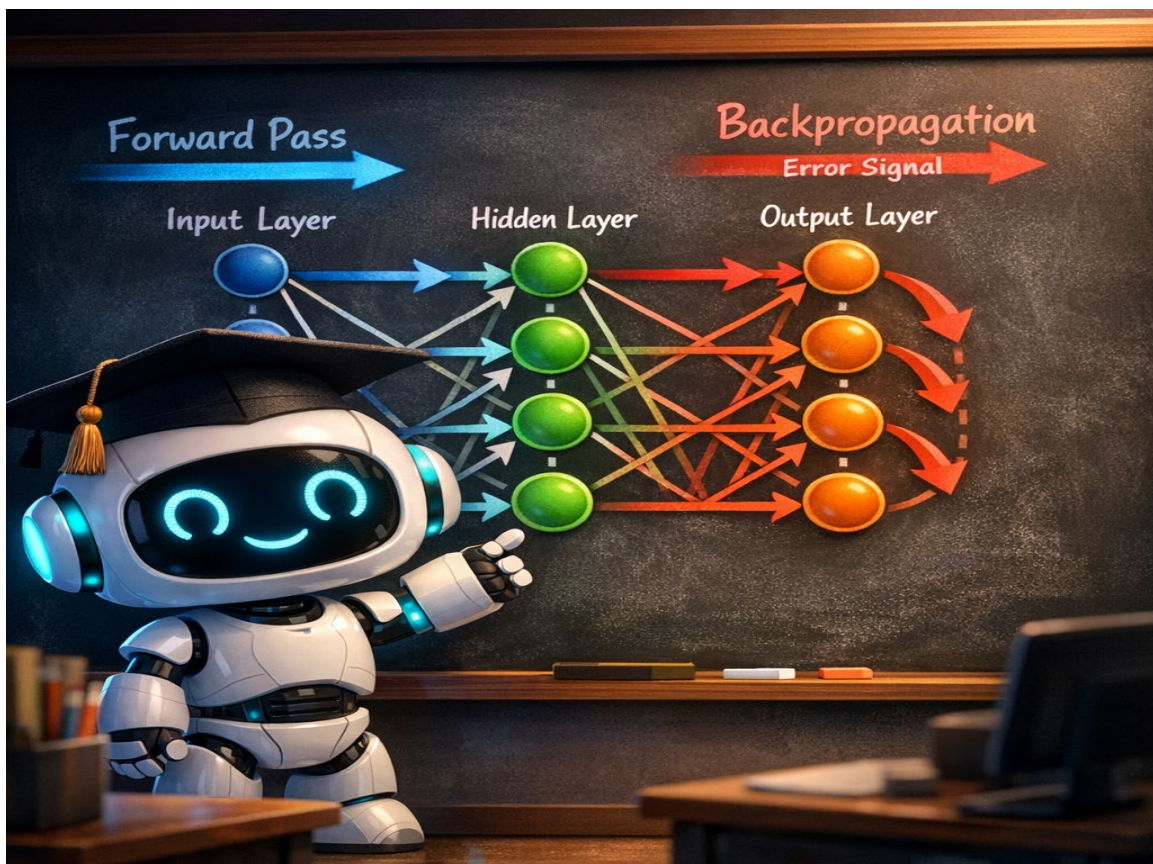
The Learning Rate

How big is each step? That is controlled by a value called the learning rate. Too big and the algorithm overshoots the valley and bounces around forever. Too small and training crawls and never finishes. Picking a sensible learning rate is the single most important hyperparameter choice in deep learning. Frameworks have built in learning rate schedulers that adjust it during training, but you still need to pick a sensible starting point.

Step 4: Backpropagation

Gradient descent tells you which direction to push the weights overall. But a neural network has thousands or millions of weights, distributed across many layers. Which specific weights should change, and by how much? That is what backpropagation answers.

Backpropagation is the algorithm that traces the error backward through the network, starting from the output and working toward the input, and tells each individual weight exactly how much it contributed to the loss. Weights that hurt the answer get reduced. Weights that helped the answer get strengthened. The math behind it is the chain rule from calculus, but you do not need to derive it. Frameworks like PyTorch and TensorFlow handle it automatically (the feature is called autograd in PyTorch, GradientTape in TensorFlow).



Historical Note

Backpropagation was originally proposed in the 1970s and rediscovered in 1986 by Rumelhart, Hinton, and Williams. It made multi layer networks finally trainable. Without it, the field would have stayed stuck on single layer perceptrons. Of all the contributions to modern AI, backpropagation is among the most important and least appreciated. It is what lets training scale to networks with billions of parameters.

Putting It All Together

Here is the whole training loop in one paragraph. The network sees a training example. It runs forward and produces a guess (forward pass). The guess is compared to the correct answer to produce a loss (loss). Backpropagation traces the error back through the network and computes how much each weight contributed (backpropagation). Gradient descent uses those numbers to update every weight, taking a small step in the direction that reduces the loss (gradient descent). Repeat with the next training example. After enough iterations, the loss drops, the predictions improve, and the network has learned.

KEY TAKEAWAY

Training a neural network is a four step loop: predict, measure error, trace blame, adjust weights. Forward pass, loss, backpropagation, gradient descent. Frameworks handle the math; engineers tune the hyperparameters and read the training curves.

THINK ABOUT IT | What if the loss never goes down?

If you start training and the loss curve is flat for an hour, something is wrong. The network is not learning. Three things to check immediately: is the learning rate too low (the steps are so small that nothing moves), is the learning rate too high (the steps keep overshooting), or is the data corrupted (labels do not match images). Reading the loss curve early and often is what separates engineers who debug fast from engineers who burn weeks on broken runs.

MISCONCEPTION ALERT | Backpropagation is not magic

Newcomers sometimes treat backpropagation as a black box that 'somehow' makes the network learn. It is not magic. It is calculus applied to a specific structure. The chain rule, taught in any calculus class, is what powers backpropagation. The framework computes it for you so you do not have to. The math exists. Knowing it exists, even without doing it by hand, keeps the system from feeling mysterious.

 INDUSTRY PRACTICE | Tuning the learning rate is a daily skill

Every CV engineer tunes learning rates dozens of times in a career. The standard practice is to start with a value the framework recommends (often 0.001 for the Adam optimizer), watch the loss curve, and adjust. If the loss explodes, halve the rate. If the loss is flat, double it. Modern frameworks include learning rate schedulers that adjust the rate automatically over training. Knowing how to read the curve and react is more valuable than memorizing any specific number.

 KNOWLEDGE CHECK | Quick check on the training loop

1. Name the four steps of the training loop in order.
2. What is the difference between the loss function and gradient descent?
3. Why does the framework handle backpropagation automatically rather than asking you to compute it?

Part 04 | Activation Functions

Each neuron computes a weighted sum of its inputs. But what happens after that sum is computed? The neuron does not just pass the raw sum forward. Instead, it passes the sum through an activation function, which decides what value to actually output. Without activation functions, a neural network would collapse into a single linear function regardless of how many layers it had. The activation function is what gives the whole system its power to learn complex patterns.

Why Activation Functions Matter

Imagine stacking ten layers of perceptrons but without any activation function. Layer one would just multiply and add. Layer two would do the same. Layer three. Mathematically, the whole stack would still be equivalent to a single layer doing one big multiplication and one big addition. The depth would be wasted.

Activation functions add non linearity. Each non linear step lets the next layer combine the previous layer's outputs in ways that would not be possible with a linear function. Stack enough non linear layers and the network can approximate any pattern in the data. The math result that proves this is called the universal approximation theorem, and it dates back to 1989.

The Four Activation Functions You Must Know

In practice, four activation functions cover the vast majority of cases. You will see all of them in framework documentation, papers, and code. Know them by name.

ReLU (Rectified Linear Unit)

ReLU is the workhorse of modern deep learning. Its rule is simple: if the input is positive, output it unchanged. If the input is negative, output zero. That is the entire function. ReLU is fast (it requires no math beyond a comparison), it avoids the vanishing gradient problem that plagued earlier activations, and it usually trains networks faster than alternatives. The downside is that ReLU can suffer from 'dead neurons' (neurons that always output zero and stop learning). Variants like Leaky ReLU and ELU were invented to fix that.

Sigmoid

Sigmoid squashes any input value into the range zero to one, producing a smooth S curve. It is most often used in the output layer of a binary classifier, where the output represents a probability (zero percent to one hundred percent). Sigmoid was the dominant hidden layer activation in the 1990s but has been largely

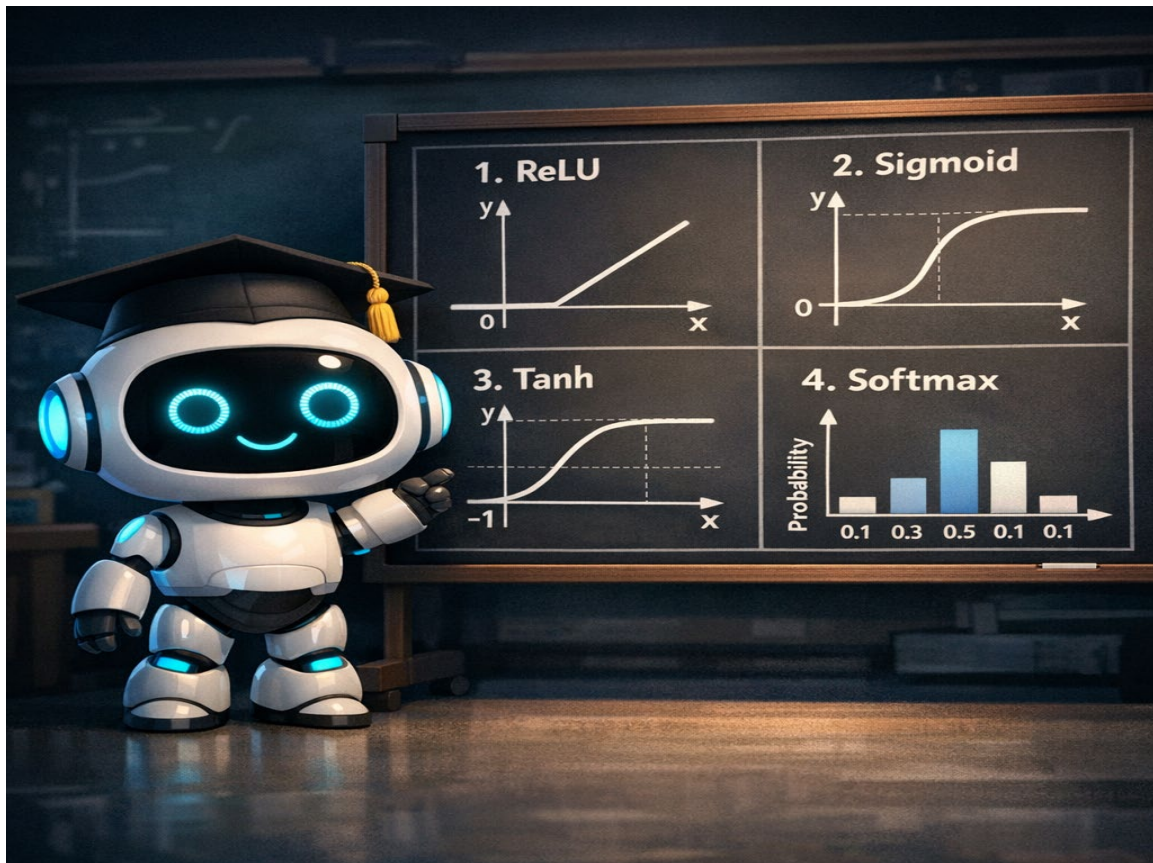
replaced by ReLU because of its tendency to slow training by producing tiny gradients (the vanishing gradient problem).

Tanh

Tanh is similar to Sigmoid but produces values in the range minus one to plus one rather than zero to one. The shape is the same S curve, just shifted. Tanh was the standard hidden layer activation in early networks for image recognition. Today it appears mostly in older architectures or as the recurrent activation in some specialized models.

Softmax

Softmax is the standard activation for the output layer of a multi class classifier. It takes a list of raw scores from the output neurons and converts them into probabilities that add up to one. For example, an output of '5.2, 3.1, 0.7' for a three class problem might get converted to '0.78, 0.16, 0.06' as probabilities. Softmax never appears in hidden layers; it is a final step only.



Choosing an Activation

For modern hidden layers, default to ReLU. It is the most common choice and almost always works. If you notice many neurons dying, switch to Leaky ReLU. For the output layer of a binary classifier, use Sigmoid. For the output layer of a multi class classifier, use Softmax. For regression problems (predicting a number), use no activation on the output (or a linear activation, which is the same thing).

INDUSTRY PRACTICE | The default rule

If you are building a new network and unsure which activation to use, follow this rule. Hidden layers: ReLU. Binary output: Sigmoid. Multi class output: Softmax. Regression output: nothing. This rule covers about ninety percent of practical cases and gets you to a working baseline fast.

MATH MOMENT (optional) | Non linearity in plain English

A linear function is anything you can describe with the word 'proportional'. Double the input, double the output. Add two inputs, add the outputs. Stack two linear functions and you get another linear function. A non linear function breaks that pattern. Doubling the input might do nothing (if the input was already large enough to saturate) or might double the output (if you are in a linear region). Activation functions add this break point. Without it, the entire network would behave like a single linear function regardless of depth. With it, the network can model curves, switches, and complex patterns.

MISCONCEPTION ALERT | More activations are not better

Some newcomers think that mixing many activation functions makes a network more powerful. It does not. Mixing activations across layers usually hurts more than it helps because it complicates training. The rule is simple: pick one activation for hidden layers and use it consistently. ReLU for almost everything. Sigmoid or Softmax only for the output layer when the task demands it. Restraint is part of the design.

 KNOWLEDGE CHECK | Quick check on activations

1. Why is ReLU the default for hidden layers in modern networks?
2. A network predicts whether a tumor is malignant or benign. Which activation function should the output layer use?
3. What is the role of an activation function in a neural network? What happens if you remove all of them?

Part 05 | Where MLPs Hit Their Limits

A multi layer perceptron is the foundation of modern deep learning. It can in principle approximate any function, classify anything, regress anything. But in practice, basic MLPs run into trouble when the inputs are images. This part explains why, and previews the architecture (the convolutional neural network) that solved the problem and that you will study in Module 05.

Problem 1: Way Too Many Weights

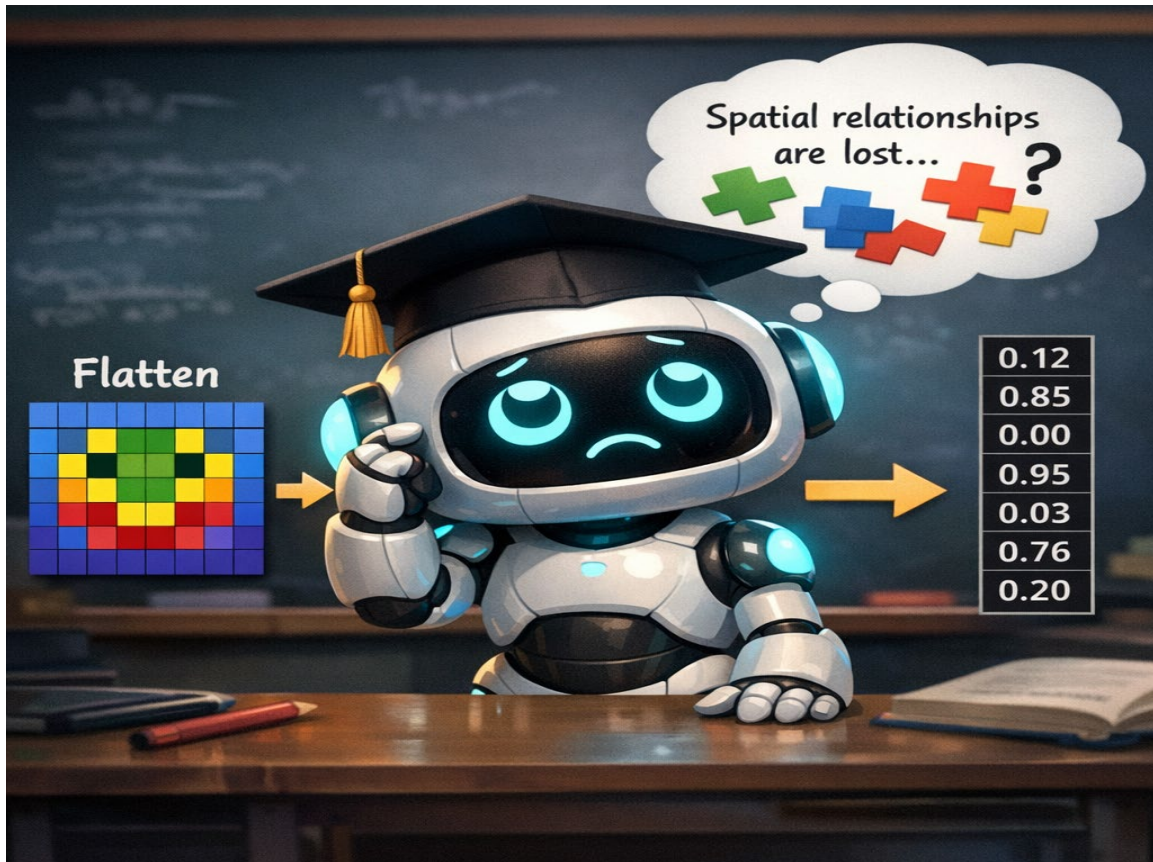
Consider a modest color image: 224 pixels wide, 224 pixels tall, three color channels (red, green, blue). That is over 150,000 pixels per image. To feed it into an MLP, you flatten the whole image into a list of 150,000 numbers. If the first hidden layer has one thousand neurons, every one of those neurons connects to every one of those 150,000 inputs, with its own weight. That is 150 million weights in just the first layer. Train such a network and it will be slow, memory hungry, and prone to overfitting.

The fundamental problem is that an MLP treats every input pixel as if it were independent of every other pixel. It has no built in notion that adjacent pixels are related, or that the same pattern can appear anywhere in the image. So it has to learn from scratch, with sheer brute force parameter count, what it should know about spatial structure for free.

Problem 2: No Sense of Location

A second consequence is that an MLP has no idea where in the image something is. A cat in the top left corner produces a totally different pattern of pixel activations than the same cat in the bottom right. The MLP sees these as unrelated inputs and has to learn separately to recognize a cat in every possible location, scale, and orientation. The network ends up with thousands of redundant detectors, each one specialized to a slightly different position.

Real images have a property that an MLP cannot exploit: translation invariance. A cat is a cat whether it is on the left or the right. A wheel is a wheel anywhere on a car. A defect is a defect at any position on a circuit board. The right architecture for image data should bake this property in by design.



The Path Forward: Convolutional Neural Networks

The convolutional neural network, or CNN, is the architecture that solved the image problem. CNNs are still neural networks, still trained the same way (forward pass, loss, gradient descent, backpropagation), still made of artificial neurons. The difference is two clever design tricks that bake image structure into the architecture itself.

Trick 1: Look at Small Patches

Instead of every pixel connecting to every neuron, a CNN slides small filters across the image, looking at neighborhoods of nearby pixels rather than the whole picture at once. A typical filter might be three by three pixels in size. This drastically reduces the number of weights and forces the network to look at local patterns, which is what real visual features (edges, corners, textures) actually are.

Trick 2: Share Filters Everywhere

A filter that detects edges in the top left of the image should also detect edges in the bottom right. So a CNN uses the same filter at every position. One filter, one set of weights, applied everywhere. This is called

weight sharing, and it gives the network translation invariance for free. It also reduces the parameter count by another large factor.

Module 05 dives into how CNNs actually work, layer by layer. For now, just know that the two limits described in this part (too many weights, no spatial awareness) are exactly the two problems CNNs were designed to fix. Everything you learned about training (forward pass, loss, gradient descent, backpropagation, activation functions) carries forward to CNNs without change. Only the architecture differs.

KEY TAKEAWAY

Basic neural networks struggle with images for two reasons: too many weights, and no sense of where pixels are relative to each other. Convolutional neural networks fix both by looking at small local patches and reusing the same filters everywhere in the image. Module 05 will dive into the details.

INDUSTRY PRACTICE | MLPs still ship for non image data

Do not write off the basic MLP. While CNNs took over image work, MLPs are still the right architecture for tabular data (rows of numbers with no spatial structure), simple time series, embedded systems where memory is precious, and the final classifier head on top of a CNN feature extractor. Many production systems use a CNN to turn images into features and then a small MLP to make the final decision. Knowing both is what makes you flexible.

KNOWLEDGE CHECK | Quick check on MLP limits

1. An MLP processes a single 224 by 224 color image. Roughly how many input numbers does it see, and why is that a problem?
2. What does translation invariance mean, and why does an MLP not have it?
3. Name the two structural tricks a CNN uses to fix the problems an MLP has with images.

Part 06 | Frameworks and Workforce Skills

Knowing the math is one thing. Knowing the tools is another. This part covers the practical software you will actually use as a CV engineer, plus the common training problems you will run into and how to recognize them. These are workforce skills that show up in every job interview.

The Main Frameworks

PyTorch

PyTorch is the dominant framework for research and modern CV work. Developed by Meta (Facebook), open source, free. PyTorch is famously Pythonic, which means it feels like writing regular Python rather than a special domain language. Most papers since around 2018 use PyTorch. If you start a new CV project in 2026, PyTorch is the default choice unless you have a strong reason to use something else.

TensorFlow and Keras

TensorFlow is the framework developed by Google. Keras is the high level API that sits on top of TensorFlow and makes it easier to use. TensorFlow is heavily used in production environments, in mobile and edge deployments (through TensorFlow Lite), and in many Google products. It has a strong reputation for production tooling. Many CV jobs in industry still list TensorFlow or Keras as a requirement, especially in companies that adopted it before PyTorch became dominant.

Hugging Face Transformers

Hugging Face is a library and a community hub for pre trained models. Most modern CV projects do not train networks from scratch; they start from a pre trained model and fine tune it on the task at hand. Hugging Face hosts thousands of pre trained models that you can download with a single line of code. Vision transformers, image classification models, segmentation models, detection models, multimodal models, all available.

ONNX (Open Neural Network Exchange)

ONNX is the standard format for moving trained models between frameworks. Train a model in PyTorch, export it to ONNX, then run inference in production using a different runtime. ONNX is critical for deployment, especially when the production environment is different from the development environment. Edge devices, mobile apps, and many cloud inference services use ONNX.

** CAREER CONNECTION | Listed in every CV job description**

Open any computer vision job listing today and you will see PyTorch listed as a requirement, with TensorFlow as a frequent addition. Hugging Face appears in any job that involves modern models. ONNX shows up in any role that involves production deployment. Know these four names cold and be able to explain what each one does.

Common Training Problems

Almost every neural network project runs into the same set of issues. Recognizing them early, by reading the training loss curve, is what separates engineers who ship from engineers who get stuck.

Overfitting

The network memorizes the training set instead of learning general patterns. Symptom: training accuracy is excellent (95 percent or higher), but validation accuracy is much worse (60 to 70 percent). The model performs poorly on data it has not seen. Fix: collect more training data, simplify the network, add regularization (dropout, weight decay), or use data augmentation to make the training set more varied.

Underfitting

The network is not powerful enough to learn the problem. Symptom: training accuracy and validation accuracy are both poor. The model is not even learning the training set well. Fix: make the network bigger (more layers, more neurons), train for longer, use better features, or revisit the data quality.

Vanishing Gradients

Gradients become so tiny that early layers stop learning. Common in deep networks built with old activation functions like Sigmoid. Symptom: training stalls after a few epochs, with early layers showing essentially zero change. Fix: switch to ReLU activations, use better weight initialization, add residual connections, or reduce network depth if the architecture allows.

Bad Learning Rate

Too high a learning rate causes the loss to explode or oscillate wildly. Too low a learning rate causes training to crawl or never reach a good solution. Symptom on the loss curve: spikes upward (too high) or a flat line near the starting value (too low). Fix: use a learning rate scheduler that decreases the rate over time, or simply try several values and pick what works.

Class Imbalance

One class has far more examples than another, and the network learns to always predict the majority class. Symptom: high overall accuracy but terrible accuracy on the minority class. Common in medical imaging (most scans are healthy) and fraud detection (most transactions are legitimate). Fix: weight the loss function so the minority class counts more, or oversample the minority class during training.

INDUSTRY PRACTICE | Reading the loss curve in one minute

1. Loss going up over time: learning rate is too high. Reduce it.
2. Loss going down then plateauing immediately: learning rate is too low or the network is too small.
3. Training loss low, validation loss high: overfitting. Add regularization or get more data.
4. Training and validation loss both stuck: underfitting. Increase capacity or check the data.
5. Loss curve looks great but accuracy is bad: probably class imbalance. Check per class metrics.

The Lab Preview

This module's hands on lab builds the classic introduction to neural networks: a digit classifier on the MNIST dataset. You will load 60,000 handwritten digits, build an MLP with two hidden layers and ReLU activations, train it for a few epochs, watch the loss drop, and evaluate it on a held out test set. By the end you will have a working neural network you built yourself, and a notebook worth saving for your portfolio.

The lab is intentionally simple. The network is small (two hidden layers, a few hundred neurons each). The dataset is tiny by modern standards. Training runs in minutes on a laptop, no GPU required. The lab is not about performance; it is about understanding. By writing every line yourself, you internalize the steps you read about in this module. After the lab, the bigger pieces (CNNs in Module 05, transfer learning in Module 06) will feel like extensions of what you already know, not new mysteries.

KEY TAKEAWAY

The four frameworks to know: PyTorch (research default), TensorFlow with Keras (production veteran), Hugging Face (pretrained models), ONNX (interchange format). The five training problems to recognize on a loss curve: overfitting, underfitting, vanishing gradients, bad learning rate, class imbalance. Master these and you will be productive in any neural network job.

 INDUSTRY PRACTICE | The dirty secret of modern CV

Almost no professional CV team trains networks from scratch anymore. The standard workflow is: download a pretrained model from Hugging Face or torchvision, swap the final layer for your specific task, fine tune on your small dataset, deploy. This is called transfer learning, and Module 06 covers it in depth. Knowing this changes how you read job descriptions: most production CV work is fine tuning, evaluation, and deployment, not pure model design.

 KNOWLEDGE CHECK | Quick check on frameworks and pitfalls

1. Match each framework to its strongest use case: PyTorch, TensorFlow with Keras, Hugging Face, ONNX.
2. A model gets 97 percent accuracy on training data and 65 percent on validation data. Diagnose the problem and name two fixes.
3. Your training loss goes up and down wildly from epoch to epoch and never decreases. What is the most likely cause?

Review Questions

Work through these on your own first. Check your answers against the booklet and your notes. If any question is uncertain, that is the section to reread.

1. Describe in plain language what a single perceptron does. Name the three operations it performs.
2. What is the main conceptual difference between classic ML (Module 03) and neural networks (this module)?
3. Name the three categories of layers in a multi layer perceptron and describe what each one does.
4. Walk through the four steps of the training loop, naming each step.
5. Why do we need activation functions at all? What would a network without them be equivalent to?
6. Pick the right activation function for each layer: (a) hidden layer of an image classifier, (b) output layer of a yes or no medical diagnosis, (c) output layer of a 10 class digit classifier, (d) output layer of a regression model predicting house price.
7. List two specific reasons a basic MLP struggles with images, and name the architecture that fixes both problems.
8. Name the four main frameworks for neural network development and describe what each one is best at.
9. You train a network and see training accuracy at 98 percent but validation accuracy at 62 percent. What is happening and what are two specific fixes you could try?
10. Explain in one sentence what backpropagation does. Why does the framework handle this automatically rather than the engineer doing it by hand?

Glossary

Quick reference for the terminology used in this module. Bookmark this page; the words appear in every framework documentation and every interview.

Activation Function

The rule each neuron uses to convert its weighted input sum into an output. Adds non linearity to the network. Common types: ReLU, Sigmoid, Tanh, Softmax.

Architecture

The overall shape of a neural network. How many layers, how many neurons in each, how they connect. Designed by humans, in contrast to weights and biases which are learned.

Backpropagation

The algorithm that traces the error backward through the network and tells each weight how much it contributed to the loss. Uses the chain rule from calculus. Frameworks handle it automatically.

Bias

A learned number that shifts the activation threshold of a neuron. Allows the neuron to fire when the weighted input is below or above a default.

Cross Entropy Loss

The standard loss function for classification. Heavily penalizes confident wrong predictions.

Forward Pass

Running an input through the network from input layer to output layer to produce a prediction. The first step of the training loop.

Gradient

The slope of the loss with respect to a weight. Tells the algorithm which direction and how strongly to update the weight.

Gradient Descent

The optimization algorithm that follows the gradient downhill to find weight values that minimize the loss.

Hidden Layer

Any layer between the input and the output. The transformations that turn raw inputs into useful features.

Learning Rate

The hyperparameter that controls how big a step gradient descent takes at each update. The most important hyperparameter in deep learning.

Loss Function

A function that measures how wrong the network's predictions are. The training algorithm tries to minimize the loss.

MLP (Multi Layer Perceptron)

A neural network with one or more hidden layers between input and output. The foundation of every modern deep learning model.

Neuron

The basic processing unit of a neural network. Receives inputs, applies weights and bias, passes the result through an activation function, produces an output. Also called a unit or a node.

ONNX

Open Neural Network Exchange. The standard format for moving trained models between frameworks. Critical for production deployment.

Overfitting

The condition where the network memorizes the training data and performs poorly on new data. Diagnosed by a gap between training and validation accuracy.

Perceptron

The simplest artificial neuron, invented by Frank Rosenblatt in 1958. Takes inputs, weights them, adds a bias, applies a threshold, produces an output.

PyTorch

The dominant open source framework for neural network research and modern CV development. Developed by Meta.

ReLU

Rectified Linear Unit. The most common activation function. Outputs the input if positive, zero if negative.

Sigmoid

An activation function that squashes any input into the range zero to one. Used for binary classification outputs.

Softmax

An activation function that turns a list of raw scores into probabilities that add up to one. Used for multi class classification outputs.

TensorFlow

Google's framework for neural networks. Widely used in production environments. Keras is its high level API.

Underfitting

The condition where the network is too simple to learn the problem. Both training and validation accuracy are poor.

Vanishing Gradient

A training problem where gradients become so small that early layers stop learning. Common in deep networks with Sigmoid activations.

Weight

A learned number that determines how strongly one input contributes to a neuron's output. Networks learn by adjusting weights through gradient descent.

References and Further Reading

The following are next steps for going deeper. None are required for this course, but all are excellent.

Foundational Books

Goodfellow, Ian; Bengio, Yoshua; and Courville, Aaron. Deep Learning. MIT Press, 2016. The canonical textbook for deep learning, free online at deeplearningbook.org. Mathematically rigorous; expect to spend time.

Geron, Aurelien. Hands On Machine Learning with Scikit Learn, Keras, and TensorFlow, Third Edition. O'Reilly, 2022. Practical, project oriented, and a standard recommendation for working engineers.

Elgandy, Mohamed. Deep Learning for Vision Systems. Manning Publications, 2020. An accessible 480 page introduction that covers neural network fundamentals, CNN architectures, transfer learning, GANs, and visual embeddings, all illustrated with worked CV examples. Strongly recommended as a next step after this module.

Recent Books (2024 to 2026)

Chollet, Francois. Deep Learning with Python, Third Edition. Manning Publications, 2025. The accessible introduction to deep learning by the creator of Keras, fully revised for the current era of transformers and generative models. Worked examples in TensorFlow and Keras.

Huang, Howard; Stevens, Eli; Antiga, Luca; and Viehmann, Thomas. Deep Learning with PyTorch, Second Edition. Manning Publications, 2025. The bestselling original updated by a core PyTorch developer to cover transformers, diffusion models, and generative AI alongside neural network fundamentals.

Elgandy, Mohamed. Grokking Deep Learning for Computer Vision. Manning Publications, forthcoming (MEAP available). A sequel to the author's 2020 Deep Learning for Vision Systems, focused on the modern landscape of CV through deep learning.

Ayyadevara, V Kishore and Reddy, Yeshwanth. Modern Computer Vision with PyTorch, Second Edition. Packt Publishing, June 2024 (ISBN 9781803231334). Comprehensive hands on coverage of modern CV, including CNNs, vision transformers, CLIP, Stable Diffusion, foundation models, and production deployment.

Jha, Ashish Ranjan. Mastering PyTorch, Second Edition. Packt Publishing, May 2024. Practical guide spanning CNNs to multimodal models, LLMs, mobile deployment, diffusion models, and graph neural networks. Updated for PyTorch 2.x.

Foundational Papers

Rumelhart, David; Hinton, Geoffrey; and Williams, Ronald. Learning Representations by Back Propagating Errors. Nature, 1986. The classic backpropagation paper that opened up multi layer training.

Krizhevsky, Alex; Sutskever, Ilya; and Hinton, Geoffrey. ImageNet Classification with Deep Convolutional Neural Networks. Advances in Neural Information Processing Systems, 2012. The AlexNet paper that triggered the modern deep learning era.

Cybenko, George. Approximation by Superpositions of a Sigmoidal Function. Mathematics of Control, Signals, and Systems, 1989. The universal approximation theorem.

Rosenblatt, Frank. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. Psychological Review, 1958. The original perceptron paper.

Documentation

PyTorch documentation at pytorch.org. The tutorials are excellent for getting started and the API reference is comprehensive.

TensorFlow tutorials at tensorflow.org. Keras specific guides are particularly approachable for beginners.

Hugging Face Transformers documentation at huggingface.co/docs. Strong examples for fine tuning pre trained models and using foundation models.

ONNX documentation at onnx.ai. The standard format for model interchange between frameworks.

Looking Ahead to Module 05

You leave this module with the foundation of every modern AI system. The artificial neuron, the multi layer perceptron, the training loop, the activation functions, the common pitfalls. Everything that comes next is variations on this core. Module 05 introduces the first major variation: convolutional neural networks.

CNNs were the breakthrough that made computer vision actually work at scale. The two design tricks you read about at the end of Part 05 (local patches and shared filters) get implemented in a specialized type of layer called a convolutional layer. Stack many convolutional layers and you have a CNN. Train it the same way you trained the MLP. Suddenly the network can recognize cats, detect pedestrians, segment tumors, and identify factory defects with accuracy that classic ML never approached.

In Module 05 you will learn how convolutional layers actually compute their outputs (the filter sliding across the image, multiplying and summing), why pooling layers exist (to downsample and keep what matters), what receptive field means (how big a region of the input each deep neuron eventually sees), and which classic CNN architectures defined the field (LeNet, AlexNet, VGG, GoogLeNet, ResNet). The lab will rebuild your MNIST digit classifier as a small CNN and you will see the accuracy go up while the parameter count goes down.

Everything you learned in this module carries forward unchanged. Same training loop. Same activations. Same loss functions. Same frameworks. Same pitfalls. What changes is the architecture. Watch closely how a small architectural shift produces such large practical gains.

Questions about this module? email or message me if you have any questions.